

Linguagem de Programação I - Etapa 4 - Rian Ruela Ribeiro

\$\$\$ src.controle.projeto

```
from util.persistência_arquivo import carregar_arquivo, salvar_arquivo
from entidades.empreiteiro import get_empregados, set_empregados
from entidades.equipamento import get_equipamentos, set_equipamentos
from entidades.obra import get_obras, set_obras
from entidades.contrato import get_contratos, set_contratos
from interfaces.interface_textual import loop_opções_execução
```

```
nome_arquivo = 'conjuntos_globais'
```

```
def salvar_aplicação():
```

```
    contrato_equipamentos = []
```

```
    contrato_equipamentos.append(get_empregados())
```

```
    contrato_equipamentos.append(get_equipamentos())
```

```
    contrato_equipamentos.append(get_obras())
```

```
    contrato_equipamentos.append(get_contratos())
```

```
    salvar_arquivo(nome_arquivo, objetos=contrato_equipamentos)
```

```
def recuperar_aplicação():
```

```
    contrato_equipamentos = carregar_arquivo(nome_arquivo)
```

```
    if contrato_equipamentos is not None:
```

```
        set_empregados(contrato_equipamentos[0])
```

```
        set_equipamentos(contrato_equipamentos[1])
```

```
        set_obras(contrato_equipamentos[2])
```

```
        set_contratos(contrato_equipamentos[3])
```

```
if __name__ == '__main__':
```

```
    recuperar_aplicação()
```

```
    loop_opções_execução()
```

```
    salvar_aplicação()
```

```
$$$ src.entidades.empregado
```

```
empregados = {}
```

```
def get_empregados():return empregados
```

```
def inserir_empregado(empregado):
```

```
    nome_empregado = empregado.nome
```

```
    if nome_empregado not in empregados.keys(): empregados[nome_empregado] =  
empregado
```

```
    else: print('Empregado ' + nome_empregado + ' já tem cadastro')
```

```
def set_empregados(empregado1):
```

```
    global empregados
```

```
    empregados = empregado1
```

```
class Empregado:
```

```
    def __init__(self, nome, telefone, email, endereço):
```

```
        self.nome = nome
```

```
        self.telefone = telefone
```

```
        self.email = email
```

```
        self.endereço = endereço
```

```
    def __str__(self):
```

```
        formato = '{} {:<31} {} {:<14} {} {:<32} {} {:<31} {}'
```

```
        empregado_formatado = formato.format(
```

```
            '|', self.nome,
```

```
            '|', self.telefone,
```

```
            '|', self.email,
```

```
            '|', self.endereço,
```

```
            '|'
```

```
        )
```

```
        return empregado_formatado
```

```
$$$ src.entidades.equipamento
```

```
equipamentos = {}
```

```
def get_equipamentos(): return equipamentos
```

```
def inserir_equipamento(equipamento):
```

```
    nome_equipamento = equipamento.nome
```

```
    if nome_equipamento not in equipamentos.keys():
```

```
equipamentos[nome_equipamento] = equipamento
```

```
    else: print('Equipamento ' + nome_equipamento + ' já tem cadastro')
```

```
def set_equipamentos(equipamentos1):
```

```
    global equipamentos
```

```
    equipamentos = equipamentos1
```

```
class Equipamento:
```

```
    def __init__(self, nome, valor, peso, disponível):
```

```
        self.nome = nome
```

```
        self.valor = valor
```

```
        self.peso = peso
```

```
        self.disponível = disponível
```

```
    def __str__(self):
```

```
        if self.disponível: disponível_str = 'disponível |'
```

```
        else: disponível_str = ''
```

```
        formato = '{ } {:<37} { } R$ {:<8.2f} { } {:<6}kg { } {:<22}'
```

```
        equipamento_formatado = formato.format(
```

```
            '|', self.nome,
```

```
            '|', self.valor,
```

```
            '|', self.peso,
```

```
            '|', disponível_str,
```

```
        )
```

```
        return equipamento_formatado
```

```
class Betoneira(Equipamento):
```

```
    def __init__(self, nome, valor, peso, disponível, capacidade, voltagem,  
tipo_betoneira):
```

```
        super().__init__(nome, valor, peso, disponível)
```

```
        self.capacidade = capacidade
```

```
        self.voltagem = voltagem
```

```
        self.tipo_betoneira = tipo_betoneira if tipo_betoneira in ('hobby', 'semi-
```

```
profissional', 'profissional') else 'indefinido'
```

```
def __str__(self):
    if self.disponível: disponível_str = 'disponível'
    else: disponível_str = "
formato = '{} {:.<37} {} R$ {:.<8.2f} {} {:.<6} {} {:.<17} {} {:.<20} {} {:.<10} {} {}'
betoneira_formatada = formato.format(
    '|', self.nome,
    '|', self.valor,
    '|', f'{self.peso}kg',
    '|', self.tipo_betoneira,
    '|', f'{self.voltagem}V',
    '|', f'{self.capacidade}L',
    '|', disponível_str,
)
return betoneira_formatada
```

```
class Furadeira(Equipamento):
    def __init__(self, nome, valor, peso, disponível, perfuração, aplicação):
        super().__init__(nome, valor, peso, disponível)
        self.perfuração = perfuração if perfuração in ('Metal', 'Madeira', 'Alvenaria') else
'indefinido'
        self.aplicação = aplicação
```

```
def __str__(self):
    if self.disponível: disponível_str = 'disponível'
    else: disponível_str = "
formato = '{} {:.<37} {} R$ {:.<8.2f} {} {:.<6} {} {:.<17} {} {:.<20} {} {}'
furadeira_formatada = formato.format(
    '|', self.nome,
    '|', self.valor,
    '|', f'{self.peso}kg',
    '|', self.perfuração,
    '|', self.aplicação,
    '|', disponível_str
)
return furadeira_formatada
```

```

$$$ src.entidades.obra

from entidades.equipamento import get_equipamentos

obras = {}

def get_obras():return obras

def inserir_obra(obra):
    id_obra = obra.id
    if id_obra not in obras.keys():obras[id_obra] = obra
    else:print('Obra ' + id_obra + ' já tem cadastro')

def set_obras(obras1):
    global obras
    obras = obras1

class Obra:
    def __init__(self, id, descrição, datas, execução):
        self.id = id
        self.descrição = descrição
        self.datas = datas
        self.execução = execução
        self.equipamentos = {}

    def __str__(self):
        formato = "{} {:<4} {} {:<31} {} {:<11} {} {:<9} {}"
        obra_formatado = formato.format(
            '|', self.id,
            '|', self.descrição,
            '|', self.execução,
            '|', str(self.datas),
            '|'
        )
        return obra_formatado

    def inserir_equipamentos(self, chaves_equipamentos):
        for chave in chaves_equipamentos:
            if chave not in self.equipamentos: self.equipamentos[chave] =

```

```
get_equipamentos())[chave]
    else:print('Equipamento ' + chave + ' já tem cadastro na Obra')
```

```
$$$ src.entidades.contrato
```

```
from entidades.obra import get_obras
from entidades.equipamento import Betoneira, Furadeira
from entidades.empregado import get_empregados
```

```
contratos = []
```

```
def get_contratos():return contratos
def inserir_contrato(contrato):
    if contrato not in contratos: contratos.append(contrato)
    else: print('Contrato já tem cadastro --- ' + str(contrato))
```

```
def set_contratos(contratos1):
    global contratos
    contratos = contratos1
```

```
def filtrar_contrato(data_mínima_contrato=None,
prefixo_nome_empregado=None,método_de_execução_obra=None,
peso_máximo_equipamento=None, capacidade_mínima_betoneira=None,
perfuração_furadeira=None):
    contratos_selecionados = []
    for contrato in contratos:
        if data_mínima_contrato is not None and contrato.data <
data_mínima_contrato: continue
        excluir_contrato = False
        for equipamento in contrato.obra.equipamentos.values():
            if peso_máximo_equipamento is not None and equipamento.peso >
peso_máximo_equipamento:
                excluir_contrato = True
                break
            if isinstance(equipamento, Betoneira):
                if capacidade_mínima_betoneira is not None and
equipamento.capacidade < capacidade_mínima_betoneira:
                    excluir_contrato = True
                    break
            elif isinstance(equipamento, Furadeira):
```

```

        if perfuração_furadeira is not None and equipamento.perfuração !=
perfuração_furadeira:
            excluir_contrato = True
            break
    if excluir_contrato: continue
    if prefixo_nome_empregado is not None and not
contrato.empregado.nome.startswith(prefixo_nome_empregado): continue
    if método_de_execução_obra is not None and contrato.obra.execução !=
método_de_execução_obra: continue
    contratos_selecionados.append(contrato)
return contratos_selecionados

def criar_contrato(data, id_obra, nome_empregado):
    obra = get_obras().get(id_obra)
    if obra is None:
        print('Obra ' + id_obra + ' não cadastrada')
        return
    empregado = get_empregados().get(nome_empregado)
    if empregado is None:
        print('Empregado ' + nome_empregado + ' não cadastrado na Obra ' + id_obra)
        return
    contrato = Contrato(data, obra, empregado)
    inserir_contrato(contrato)

```

```

class Contrato:
    def __init__(self, data, obra, empregado):
        self.data = data
        self.obra = obra
        self.empregado = empregado

```

```

    def __str__(self):
        formato = '{} {:<2} {} {:<31} {} {:<10} {}'
        contrato_formatado = formato.format(
            '|', self.obra.id,
            '|', self.empregado.nome,
            '|', str(self.data),
            '|')
        return contrato_formatado

```

```

    def str_atributos_equipamento(self):
        atributos_equipamento_str = ""
        for índice, equipamento in enumerate(self.obra.equipamentos.values()):

```

```
        if(indice>0): atributos_equipamento_str += ' -- '
        atributos_equipamento_str += f'{equipamento.peso}kg' + ' - '
        if isinstance(equipamento, Betoneira): atributos_equipamento_str +=
str(equipamento.capacidade) + 'L'
        elif isinstance(equipamento, Furadeira): atributos_equipamento_str +=
str(equipamento.perfuração)
        return atributos_equipamento_str

def str_filtro(self):
    formato = '{:<34} {} {:<11} {}'
    filtro_formatado = formato.format(self.str_atributos_equipamento(), '|',
self.obra.execução, '|')
    return self.__str__() + filtro_formatado
```

\$\$\$ src.util.data

```
from datetime import date
```

```
from re import match
```

```
def converte_str_para_data(data_str):
```

```
    if not match(r'(0?[1-9]|[12][0-9]|3[01])/(0?[1-9]|1[012])/^d{4}', data_str): return None
```

```
    dia, mês, ano = data_str.split('/')
```

```
    return Data(int(dia), int(mês), int(ano))
```

```
class Data:
```

```
    def __init__(self, dia, mês, ano):
```

```
        self.dia = dia
```

```
        self.mês = mês
```

```
        self.ano = ano
```

```
    def __str__(self):
```

```
        if self.dia < 10: data_str = '0' + str(self.dia)
```

```
        else: data_str = str(self.dia)
```

```
        if self.mês < 10: data_str += "/0" + str(self.mês) + "/"
```

```
        else: data_str += "/" + str(self.mês) + "/"
```

```
        data_str += str(self.ano)
```

```
        return data_str
```

```
    def __eq__(self, data):
```

```
        if self.dia == data.dia and self.mês == data.mês and self.ano == data.ano:
```

```
            return True
```

```
        return False
```

```
    def __ne__(self, data): return not self == data
```

```
    def __gt__(self, data):
```

```
        if self.ano > data.ano: return True
```

```
        elif self.ano < data.ano: return False
```

```
        if self.mês > data.mês: return True
```

```
        elif self.mês < data.mês: return False
```

```
if self.dia > data.dia: return True
elif self.dia < data.dia: return False
return False
```

```
def __lt__(self, data):
    if self.ano < data.ano: return True
    elif self.ano > data.ano: return False
    if self.mês < data.mês: return True
    elif self.mês > data.mês: return False
    if self.dia < data.dia: return True
    elif self.ano > data.ano: return False
    return False
```

```
def __ge__(self, data):
    if self < data: return False
    else: return True
```

```
def __le__(self, data):
    if self > data: return False
    else: return True
```

```
def calcular_idade(self, data_referência=None):
    if data_referência is None:
        dia_atual_str, mês_atual_str, ano_atual_str =
date.today().strftime("%d/%m/%Y").split('/')
        dia_referência, mês_referência, ano_referência = int(dia_atual_str),
int(mês_atual_str), int(ano_atual_str)
    else:
        dia_referência, mês_referência, ano_referência = data_referência.dia,
data_referência.mês, data_referência.ano
        idade = ano_referência - self.ano
        if mês_referência < self.mês or (mês_referência == self.mês and dia_referência
< self.dia):
            idade -= 1
        return idade
```

\$\$\$ src.util.gerais

```
def imprimir_objetos(cabeçalho, objetos, filtros=None):
```

```
    if filtros is not None: print(filtros)
```

```
    print(cabeçalho)
```

```
    for índice, objeto in enumerate(objetos):
```

```
        imprimir_objeto(índice, str(objeto))
```

```
def imprimir_objeto(índice, objeto_str):
```

```
    formato = '{} {} {}'
```

```
    ordem = índice + 1
```

```
    separador = '-'
```

```
    string_formatado = formato.format(f'{ordem:2d}', separador, objeto_str)
```

```
    print(string_formatado)
```

```
def imprimir_objetos_associação_filtros(cabeçalho, objetos, filtros=None):
```

```
    if filtros is not None: print(filtros)
```

```
    print(cabeçalho)
```

```
    for índice, objeto in enumerate(objetos):
```

```
        imprimir_objeto(índice, objeto.str_filtro())
```

```
def imprimir_objetos_internos(objetos):
```

```
    for objeto in objetos: print(' - ' + str(objeto))
```

```
$$$ src.util.persistência_arquivo
```

```
import pickle
```

```
def salvar_arquivo(nome_arquivo, objetos):  
    arquivo = open('../dados/' + nome_arquivo + '.bin', 'wb')  
    pickle.dump(objetos, arquivo)  
    arquivo.close()
```

```
def carregar_arquivo(nome_arquivo):  
    try:  
        arquivo = open('../dados/' + nome_arquivo + '.bin', 'rb')  
        objetos = pickle.load(arquivo)  
    except IOError: objetos = None  
    return objetos
```

```
$$$ src.interfaces.interface_textual
```

```
from util.gerais import imprimir_objetos, imprimir_objeto, imprimir_objetos_internos,
imprimir_objetos_associacao_filtros
from util.data import converte_str_para_data
from entidades.obra import Obra, inserir_obra, get_obras
from entidades.equipamento import Equipamento, inserir_equipamento,
get_equipamentos, Betoneira, Furadeira
from entidades.empreiteiro import Empreiteiro, inserir_empreiteiro, get_empreiteiros
from entidades.contrato import Contrato, filtrar_contrato, get_contratos,
criar_contrato, inserir_contrato
```

```
def loop_opções_execução():
    sair_loop = False
    cabeçalho_empreiteiro = '\nEmpreiteiro : nome - telefone - email - endereço '
    cabeçalho_equipamento = '\nEquipamento : nome - valor - peso - Betoneira [tipo -
vtagem - capacidade] | Furadeira [perfuração - aplicação] - disponível'
    cabeçalho_obra = '\nObra : id - descrição - execução - data'
    cabeçalho_obra_equipamentos = ('\nObra : id - descrição - execução - datas'
    + '\n - Equipamento : nome - tipo - valor - peso - Betoneira [vtagem -
capacidade] | Furadeira [perfuração - aplicação]'
    + ' - disponível')
    cabeçalho_contrato = '\nContrato : id da obra - nome do empreiteiro - data'
    while not sair_loop:
        print()
        operação = ler_str('Opções [C: Cadastrar / I: Imprimir / S: Selecionar / T:
Imprimir Todos / <ENTER>: Parar]', retornar=True)
        if operação == None: break
        elif operação in ('C', 'I'):
            if operação == 'I':
                opção_conteúdo = ler_str('E: Empreiteiro / Eq: Equipamento / O: Obra / C:
Contrato / EO: Equipamentos nas Obra')
```

```

        + '/' <ENTER>: retornar]', retornar=True)
    else:opção_conteúdo = ler_str('E: Empreiteiro / Eq: Equipamento / O: Obra /
C: Contrato / <ENTER>: retornar]', retornar=True)
    if opção_conteúdo == None: pass
    elif opção_conteúdo == 'E':
        if operação == 'C':loop_leitura_empreiteiros()
        imprimir_objetos(cabeçalho_empreiteiro,get_empreiteiros().values())
    elif opção_conteúdo == 'Eq':
        if operação == 'C':loop_leitura_equipamentos()
        imprimir_objetos(cabeçalho_equipamento,get_equipamentos().values())
    elif opção_conteúdo == 'O':
        if operação == 'C':loop_leitura_obras()
        imprimir_objetos(cabeçalho_obra, get_obras().values())
    elif opção_conteúdo == 'C':
        if operação == 'C':loop_leitura_contratos()
        imprimir_objetos(cabeçalho_contrato,get_contratos())
    elif operação == 'I' and opção_conteúdo == 'EO':
        imprimir_obra_equipamentos(cabeçalho_obra_equipamentos)
    elif operação == 'S': loop_seleção_contratos()
    elif operação == 'T':
        imprimir_objetos(cabeçalho_empreiteiro,get_empreiteiros().values())
        imprimir_objetos(cabeçalho_equipamento,get_equipamentos().values())
        imprimir_objetos(cabeçalho_obra, get_obras().values())
        imprimir_obra_equipamentos(cabeçalho_obra_equipamentos)
        imprimir_objetos(cabeçalho_contrato, get_contratos())
def imprimir_obra_equipamentos(cabeçalho_obra_equipamentos):
    print(cabeçalho_obra_equipamentos)
    for índice, obra in enumerate(get_obras().values()):
        imprimir_objeto(índice=índice, objeto_str=str(obra))
        imprimir_objetos_internos(obra.equipamentos.values())

def loop_leitura_empreiteiros():
    sair_loop = False
    print('--- Leitura de Dados dos Empreiteiros ---')
    while not sair_loop:
        empreiteiro = ler_empreiteiro()
        if empreiteiro is not None: inserir_empreiteiro((empreiteiro))
        else:print('-ERRO: na leitura do empreiteiro')
        sair_loop = ler_sair_loop('cadastro de empreiteiros')

def loop_leitura_equipamentos():
    sair_loop = False
    print('--- Leitura de Dados dos Equipamentos ---')

```

```
while not sair_loop:
    equipamento = ler_equipamento()
    if equipamento is not None:
        inserir_equipamento(equipamento)
    else: print('- ERRO: na leitura do equipamento')
    sair_loop = ler_sair_loop('cadastro de equipamentos')
```

```
def loop_leitura_obras():
    sair_loop = False
    print('--- Leitura de Dados das Obras ---')
    while not sair_loop:
        obra = ler_obra()
        if obra is not None:
            inserir_obra(obra)
            loop_leitura_chaves_obras_equipamento(obra)
        else: print('- ERRO: na leitura do obra')
        sair_loop = ler_sair_loop('cadastro de obra')
```

```
def loop_leitura_chaves_obras_equipamento(obra):
    print('--- Leitura dos Índices das Chaves dos Equipamentos:' + obra.id + ' ---')
    imprimir_objetos('--- Lista de índice dos equipamento',
get_equipamentos().keys())
    sequência_numeros = ler_str('Informar lista de índice dos equipamentos')
    lista_índice_chaves = sequência_numeros.split(',')
    chave_equipamentos = []
    lista_chaves = list(get_equipamentos().keys())
    for índice_str in lista_índice_chaves:
        índice = int(índice_str)
        chave_equipamentos.append(lista_chaves[índice -1])
    obra.inserir_equipamentos(chave_equipamentos)
```

```
def loop_leitura_contratos():
    sair_loop = False
    print('--- Leitura de Dados dos Contratos ---')
    while not sair_loop:
        contrato = ler_contrato()
        if contrato is not None: inserir_contrato(contrato)
        else: print(' - ERRO: na leitura do contrato')
        sair_loop = ler_sair_loop('cadastro de contrato')
```

```
def loop_seleção_contratos():
    sair_loop = False
    print('--- Seleção de Contratos ---')
```

```

while not sair_loop:
    filtros, contratos_selecionados = selecionar_contrato()
    if filtros is not None: cabeçalho = ('Contrato : id da obra - nome do empreiteiro -
data'
                                     + '\n - Equipamento : nome - tipo - valor - peso -
Betoneira [voltagem - capacidade] | Furadeira [perfuração - aplicação]'
                                     + '\n - disponível')
    imprimir_objetos_associacao_filtros(cabeçalho, contratos_selecionados, filtros)
    sair_loop = ler_sair_loop('seleção de contratos')

```

```

def ler_sair_loop(loop):
    try:
        sair = input('--sair do loop de ' + loop + ' [S]: ')
        if sair == 'S': return True
    except IOError: pass
    return False

```

```

def ler_empreiteiro():
    nome = ler_str('nome do empreiteiro')
    if nome is None: return None
    telefone = ler_str('telefone do empreiteiro')
    if telefone is None: return None
    email = ler_str('email do empreiteiro')
    if email is None: return None
    endereço = ler_str('endereço do empreiteiro')
    if endereço is None: return None
    return Empreiteiro(nome, telefone, email, endereço)

```

```

def ler_obra():
    id = ler_str('id da obra')
    if id is None: return None
    descrição = ler_str('descrição da obra')
    if descrição is None: return None
    datas = ler_data('datas da obra')
    if datas is None: return None
    execução = ler_str('execução da obra')
    if execução is None: return None
    return Obra(id, descrição, datas, execução)

```

```

def ler_equipamento():
    nome = ler_str('nome do equipamento')
    if nome == None: return None
    valor = ler_float_positivo('valor do equipamento')

```

```

if valor == None: return None
peso = ler_int_positivo('peso dos equipamento')
if peso == None: return None
disponível = ler_bool('disponível do equipamento')
if disponível == None: return None
família_equipamento = ler_str('família do equipamento [B=Betoneira /
F=Furadeira')
if família_equipamento == 'B':
    capacidade = ler_int_positivo('capacidade da Betoneira')
    if capacidade == None: return None
    voltagem = ler_str('Voltagem do Betoneira')
    if voltagem == None: return None
    tipo_betoneira = ler_tipo_betoneira()
    if tipo_betoneira == None: return None
    return
Betoneira(nome,valor,peso,disponível,capacidade,voltagem,tipo_betoneira)
if família_equipamento == 'F':
    perfuração = ler_perfuração_furadeira('perfuração da Furadeira')
    if perfuração == None: return None
    aplicação = ler_str('Aplicação da Furadeira')
    if aplicação == None: return None
    return Furadeira(nome, valor, peso, disponível, perfuração, aplicação)
else: return None

def ler_contrato():
    data = ler_data('data do contrato')
    if data is None: return None
    id_obra = ler_str('id da obra')
    if id_obra is None: return None
    nome_empregado = ler_str('nome do empregado')
    if nome_empregado is None: return None
    return criar_contrato(data, id_obra, nome_empregado)

def selecionar_contrato():
    filtros = '\nFiltros -- '
    data_mínima_contrato = ler_data('data mínima do contrato ',filtro=True)
    if data_mínima_contrato is not None: filtros += 'data mínima do contrato: ' +
str(data_mínima_contrato)
    prefixo_nome_empregado = ler_prefixo_nome_empregado(filtro=True)
    if prefixo_nome_empregado is not None: filtros += '- prefixo do nome empregado: ' +
str(prefixo_nome_empregado)
    método_de_execução_obra = ler_obra_execução('método de execução obra ',
filtro=True)

```

```

    if método_de_execução_obra is not None: filtros += ' - ' +
método_de_execução_obra
    peso_máximo_equipamento = ler_int_positivo('peso do equipamento ', filtro=True)
    if peso_máximo_equipamento is not None: filtros += ('- peso máximo dos
equipamentos: ' + str(peso_máximo_equipamento))
    capacidade_mínima_betoneira = ler_int_positivo('capacidade da betoneira ',
filtro=True)
    if capacidade_mínima_betoneira is not None: filtros += ' - capacidade mínima da
betoneira: ' + str(capacidade_mínima_betoneira)
    perfuração_furadeira = ler_perfuração_furadeira('perfuração da furadeira ',
filtro=True)
    if perfuração_furadeira is not None: filtros += '\n- perfuração da furadeira ' +
str(perfuração_furadeira)
    contratos_selecionados = filtrar_contrato(data_mínima_contrato,
prefixo_nome_empregado, método_de_execução_obra,
peso_máximo_equipamento, capacidade_mínima_betoneira, perfuração_furadeira)

    return filtros, contratos_selecionados

```

```

def ler_str(dado, filtro=False, retornar=False):
    try:
        string = input('- ' + dado + ' : ')
        if len(string) == 0 and (filtro or retornar): return None
        if len(string) > 0: return string
    except IOError: pass
    print('Erro na leitura do dado: ' + dado)
    return None

```

```

def ler_int_positivo(dado, filtro=False):
    try:
        string = input('- ' + dado + ' : ')
        if len(string) == 0 and filtro: return None
        int_positivo = int(string)
        if int_positivo > 0: return int_positivo
    except ValueError: pass
    print('Erro na leitura/conversão do inteiro positivo: ' + dado)
    return None

```

```

def ler_float_positivo(dado, filtro=False):
    try:
        string = input('- ' + dado + ' : ')
        if len(string) == 0 and filtro: return None
        float_positivo = float(string)

```

```
    if float_positivo > 0.0: return float_positivo
except ValueError: pass
print('Erro na leitura/conversão do flutuante positivo: ' + dado)
return None
```

```
def ler_bool(dado, filtro=False):
    try:
        string = input('- ' + dado + ' [S/N]: ')
        if len(string) == 0 and filtro: return None
        if string == 'S': return True
        elif string == 'N': return False
    except ValueError: pass
    print('Erro na leitura do booleano: ' + dado)
    return None
```

```
def ler_data(dado, filtro=False):
    try:
        string = input('- ' + dado + ' [dd/mm/aaaa]: ')
        if len(string) == 0 and filtro: return None
        data = converte_str_para_data(string)
        if data is not None: return data
    except IOError: pass
    print('Erro na leitura da data: ' + dado)
    return None
```

```
def ler_prefixo_nome_empregado(filtro=False):
    try:
        string = input(' - prefixo empregado [C=Construções / S=Serviço]: ')
        if len(string) == 0 and filtro: return None
        if string == 'C': return 'Construções'
        if string == 'S': return 'Serviços'
    except ValueError: pass
    print('Erro na leitura do prefixo empregado')
    return None
```

```
def ler_obra_execução(dado, filtro=False):
    try:
        string = input(' - execução da obra [H=Horizontal / V=Vertical]: ')
        if len(string) == 0 and filtro: return None
        if string == 'H': return 'Horizontal'
        if string == 'V': return 'Vertical'
    except ValueError: pass
    print('Erro na leitura da execução da obra')
```

```
return None
```

```
def ler_tipo_betoneira(filtro=False):  
    try:  
        string = input(' - tipo da Betoneira [H=hobby / S=semi-profissional /  
P=profissional]: ')  
        if len(string) == 0 and filtro: return None  
        if string == 'H': return 'hobby'  
        if string == 'S': return 'semi-profissional'  
        if string == 'P': return 'profissional'  
    except ValueError:  
        pass  
    print('Erro na leitura do tipo da betoneira')  
    return None
```

```
def ler_perfuração_furadeira(dado,filtro=False):  
    try:  
        string = input(' - perfuração da Furadeira [M=Metall / Ma=Madeira /  
A=Alvenaria]: ')  
        if len(string) == 0 and filtro: return None  
        if string == 'M': return 'Metal'  
        if string == 'Ma': return 'Madeira'  
        if string == 'A': return 'Alvenaria'  
    except ValueError:  
        pass  
    print('Erro na leitura da perfuração da furadeira')  
    return None
```

```
def ler_peso Equipamento(filtro=False):  
    try:  
        string = input(' - peso máximo do equipamento (kg): ')  
        if len(string) == 0 and filtro: return None  
        peso = float(string)  
        if peso > 0: return peso  
    except ValueError:  
        pass  
    print('Erro na leitura do peso do equipamento')  
    return None
```

Dourados, 16/06/2026

Rian Ruela Ribeiro



Documento assinado digitalmente

RIAN RUELA RIBEIRO

Data: 16/06/2026 23:02:16-0300

Verifique em <https://validar.iti.gov.br>